

check_cm_no_fail_open_skip

check_cm_no_fail_open_skip.sh — CM-NO-FAIL-OPEN-SKIP

Revision: 1 **Last modified:** 2026-09-23T12:00:00Z **Authority:** §11.4.69 (names this gate as mandatory) · §11.4.3 (topology SKIP-with-reason) · §11.4.201 (guards assert the real condition; control needle) · §11.4.135 / §11.4.227(A) (monotone ratchet) · §11.4.18 (script documentation)

Scope: pre-build invariant 57, scripts/pre_build/check_cm_no_fail_open_skip.sh, its engine scripts/pre_build/cm_no_fail_open_skip_analyzer.py, and its baseline scripts/pre_build/cm_no_fail_open_skip.baseline **Tracker:** BOB-161 (the gate) · BOB-192 (remediating the baselined findings)

Overview

A **fail-open skip** is a test that sees evidence the far side *answered* — an HTTP status, an empty or incomplete body, an exception that also carries answered HTTP errors — and turns it into a SKIP. Every summary a human reads counts a skip alongside a pass (“N passed, M skipped, 0 failed”), so a failure the product really exhibited leaves the run green.

This gate scans every test under tests/ and fails the build when a new fail-open skip appears. It exists because BOB-092 removed two of these from tests/e2e/test_live_stack_evidence.py one at a time, both found by an agent reading code rather than by the automated regime (a §11.4.238 coverage escape). The guards written then live in the file they guard; this gate covers the whole corpus.

What is NOT a finding: a skip on *environment* evidence — a missing binary (`shutil.which`), unset credentials (`os.environ`), a missing file, a TCP port that refuses connections, or an except that catches **only** connection-class errors (`ConnectionError`, `ConnectError`, `Timeout`). Those mean the topology is absent, which is the legitimate §11.4.3 SKIP.

Prerequisites

- `bash`, `git`, `python3` (standard library only — no `venv` needed).
- The root passed with `--root` must be a git work tree.

Usage

```
# Normal gate run (what pre-build invariant 57 does)
bash scripts/pre_build/check_cm_no_fail_open_skip.sh

# Print the current finding SET (use when you tighten the
  baseline)
bash scripts/pre_build/check_cm_no_fail_open_skip.sh --list

# Scan another tree against another baseline (the self-test does
  this)
bash scripts/pre_build/check_cm_no_fail_open_skip.sh --root /path/
  to/tree --baseline /path/to/file

# Self-test (golden-TRUE, golden-FALSE with carriers, ratchet,
  blind, §1.1 mutations)
bash tests/pre_build/test_check_cm_no_fail_open_skip.sh
```

Exit codes: 0 PASS (finding SET equals the baseline SET) · 1 FAIL (NEW finding or STALE baseline row) · 2 ERROR (the tree is UNVERIFIED — see below).

The detection rule

A *skip site* is `pytest.skip(...)`, a bare `skip(...)`, `<x>.skipTest(...)`, or `raise [unittest.]SkipTest(...)`. `@pytest.mark.skip` decorators are not runtime skips and are not counted. A skip site is a finding when, inside the same function, it is enclosed by:

Trigger	Enclosing construct	Example
STATUS	an <code>if/elif/while/ternary</code> test that reads a status of response-derived data —	<code>if resp.status >= 400:</code> <code>pytest.skip(...)</code>

Trigger	Enclosing construct	Example
EMPTY	<code>.status, .status_code, .ok, .code, .reason,</code> <code>a</code> <code>["status"] / .get("error_type")-style</code> <code>key, or a comparison with</code> <code>an int literal in 100..599</code> <code>such a test that checks</code> <code>response-derived data for</code> <code>absence — not x, x is None,</code> <code>len(x) == 0, x == "", "k"</code> <code>not in x, or bare truthiness</code> <code>of x</code>	<code>if not data["results"]:</code> <code>pytest.skip(...)</code>
UNREACH	<code>an except whose try</code> <code>performs a network call and</code> <code>whose caught types include</code> <code>one that ALSO carries</code> <code>answered errors —</code> <code>HTTPError, URLError,</code> <code>RequestException, OSError,</code> <code>Exception, bare except</code>	<code>except</code> <code>urllib.error.URLError:</code> <code>pytest.skip(...)</code>

`urllib.request.urlopen` raises `HTTPError` — a subclass of `URLError` — for every 4xx/5xx response, so `except URLError` turns an answered 500 into a skip. That is why `UNREACH` is keyed on what the handler *can* catch, not on its intent.

Response-derived means intra-module taint: a name bound from a network call (`requests.*`, `httpx.*`, `urlopen`, `<client>.get ...`), from a call to a module function that transitively performs one, or from an expression over an already tainted name.

Shell tests: a bare `ab_skip` at command position is a finding (`SHELL_BARE_SKIP`); so is `ab_skip_with_reason` whose reason is outside the §11.4.69 closed set or is `network_unreachable_external` (`SHELL_REASON` — forbidden for sink-probed features, and the feature class is not statically knowable, so it is refused conservatively).

Carriers are never findings. Comments, strings, docstrings and heredoc bodies that merely *mention* a skip are ignored (Python is parsed with `ast`; shell strings, comments and heredocs are blanked before matching). The free-text reason of a `pytest skip` and any `# allow-skip: comment` never excuse a finding.

Keys and the baseline

Each finding has a key <path>:<qualname>:<TRIGGERS>#<n>, where n numbers the findings of that trigger inside that function in line order. Keys, not line numbers, so editing an unrelated part of the file does not churn the baseline.

The baseline is compared as a **set**, in both directions:

- a live finding not in the baseline is **NEW** → FAIL. Fix the test; never add a row to silence it.
- a baseline row with no live finding is **STALE** → FAIL. Delete the row in the same change that fixes the finding, so the bar follows the corpus down.

A count-only baseline would accept a one-out-one-in swap; the set comparison does not (self-test case 6d).

Baseline as seeded on 2026-09-23 (10 findings, all tracked by BOB-192)

Key

tests/integration/test_jackett_autoconfig_real.py:jackett_ready:STATUS#1

tests/integration/test_jackett_autoconfig_real.py:jackett_ready:UNREACH#1

tests/integration/
test_merge_api.py:TestDownloadEndpoint.test_download_magnet_added_to_real_qbittor

tests/integration/
test_merge_api.py:TestSearchEndpoint.test_search_finds_real_results_for_common_qu

tests/integration/test_merge_api.py:_qbit_login:STATUS#1

tests/integration/test_tracker_auth_live.py:_merge_service_required:STATUS#1

tests/integration/test_tracker_auth_live.py:_merge_service_required:EMPTY#1

Key

tests/integration/test_tracker_auth_live.py:_merge_service_required:UNREACH#1

tests/integration/test_tracker_auth_live.py:_run_live_search:EMPTY#1

tests/scaling/test_boba_scaling.py:_services_up:EMPTY#1

Edge cases and the ERROR (rc=2) states

The gate refuses rather than reporting a PASS it cannot back:

- **Control needle not seen.** Before scanning, the engine runs built-in canaries through the same functions: one golden-TRUE per trigger class plus a shell bare `ab_skip` must all be found, and a golden-FALSE canary full of carriers and environment-derived skips must yield zero findings while all of its real skip sites are still counted. If any canary misbehaves the instrument is blind (§11.4.201(7)(b)). A PASS line ends with `control-needle: seen`.
- **Blind zero.** Zero skip sites across the whole corpus is refused: a broken extractor and a clean corpus look identical (§11.4.201(6)).
- **Unparseable test file.** The file is named, and the tree is UNVERIFIED, not clean.
- **Not a git tree / python3 missing / engine missing.**

Internal behaviour

1. The wrapper checks its preconditions and runs the engine.
2. The engine runs the control needle.

3. It enumerates `tests/**/*.py` and `tests/**/*.sh` with `git ls-files --cached --others --exclude-standard`, so a new test file that is not yet staged is scanned too, and git-ignored paths (for example agent worktrees) are not.
4. Python files are parsed with `ast`; each function gets a taint set; each skip site's enclosing conditions and exception handlers are classified.
5. Findings are keyed, compared against the baseline set, and a one-line verdict is printed last.

Honest limits

- Taint does not cross module boundaries or pytest fixture parameters. A skip on data a fixture fetched in another file (for example `tests/integration/test_iporrents.py` line 160) is not seen.
- Two same-trigger skips swapped inside one function keep the same key set.
- The UNREACH rule needs the network call to appear in the `try` body (directly or via a module-local helper). Wrappers around `request.getfixturevalue(...)` are not flagged; they depend on the fixture raising only on real unreachability.
- Shell coverage is limited to the `ab_skip` family; a shell test's own ad-hoc skip helper conditioned on a `curl` status is not analysed.

Related scripts

- `scripts/pre_build_verification.sh` — invariant 57 runs this gate (blocking).
- `tests/pre_build/test_check_cm_no_fail_open_skip.sh` — the self-test, also run by invariant 30.
- `scripts/pre_build/check_cm_sse_route_rate_limit_classed.sh` — same wrapper-plus-AST-engine pattern.

Last verified

2026-09-23 — self-test all cases PASS; real tree PASS with 10 baselined findings, 69 skip sites across 412 test files.